

Equity Derivatives Pricing & Risk Workbench — Plan détaillé (v2)

Projet portfolio Python — objectif : stage de 5 mois en equity derivatives (banque d'investissement), à partir d'avril 2027.

Objectif du projet

Construire, seul et de bout en bout, un petit outil de pricing et de risque sur options vanille et autocall, pour deux raisons :

- avoir un sujet solide et défendable en entretien technique ;
- montrer une motivation et une initiative concrètes, au-delà des notes et du CV.

Le projet reste volontairement contenu : pas de vol stochastique, pas de calibration de surface de vol, pas de courbe de taux — ces limites sont assumées et même valorisées dans le README (savoir où s'arrête son modèle est un signe de maturité).

Méthode de travail

Pour chaque phase : je t'explique la théorie et la logique, tu écris le code toi-même dans ton style, je te pose des questions de vérification avant qu'on avance. Je ne donne pas de solution "clé en main".

Un point important : le meilleur moyen que ce projet ait l'air (et *soit*) fait par toi, ce n'est pas de fabriquer des signes d'authenticité, c'est de faire réellement le travail toi-même, sur tes vraies sessions — les hésitations, le nommage perso, les bugs corrigés viendront alors naturellement, sans mise en scène.

Architecture finale

equity-derivatives-workbench/

```
├── pricing/
│   ├── black_scholes.py
│   ├── implied_vol.py
│   └── payoff.py
├── greeks/
│   └── greeks.py
├── monte_carlo/
│   └── autocall.py
├── risk/
│   ├── scenarios.py
│   └── pnl_explain.py
├── tests/
│   └── test_black_scholes.py
```

```
|
| └─ test_greeks.py
|   └─ test_autocall.py
|
| └─ app.py
|   └─ requirements.txt
|   └─ README.md
```

Chaque dossier correspond à une compétence à savoir expliquer séparément en entretien. On ne mélange jamais pricing, Greeks, simulation et interface dans un seul fichier.

Changement par rapport à la v1 : les tests ne sont plus écrits en bloc à la fin (ancienne Phase 7), mais juste après chaque fonctionnalité — c'est à la fois une meilleure pratique et un historique Git plus naturel. La Phase 7 devient une phase de consolidation légère.

Phase 0 — Fondations

Objectif : un environnement propre, un dépôt Git qui vivra dans le temps.

1. Créer le dossier du projet, `git init`.
 2. Créer un environnement virtuel (`python -m venv venv`).
 3. Installer les librairies au fur et à mesure des besoins — on commence avec `numpy` et `scipy`, le reste viendra plus tard.
 4. Premier commit : structure de dossiers vide + `README.md` avec juste un titre et une phrase d'intention.
 5. Les commits suivants s'étaleront sur tes vraies sessions de travail, avec des messages type "ajout fonction black scholes call", "fix erreur signe d1 d2".
-

Phase 1 — Pricing Black-Scholes + volatilité implicite

Objectif : coder toi-même, à la main, le pricing d'une option vanille — et sa réciproque.

Contenu de `pricing/black_scholes.py` :

1. Une fonction qui calcule `d1` et `d2` à partir de `spot`, `strike`, `maturité`, `volatilité`, `taux` et `dividende`.
2. Une fonction `prix_call` et une fonction `prix_put`.
3. Vérification manuelle sur papier/Excel d'un cas simple (`spot=100`, `strike=100`, `T=1 an`, `vol=20%`, `taux=2%`), comparée au résultat Python.

Ajout par rapport à la v1 — `pricing/IMPLIED_VOL.py` : Une fonction qui fait l'inverse : à partir d'un prix d'option observé, retrouver la volatilité implicite (via `scipy.optimize.brentq` ou Newton-Raphson maison). C'est peu de code, mais c'est une brique quasi incontournable en entretien equity derivatives — le marché parle en vol implicite, pas en vol historique.

Test immédiat : écris `tests/test_black_scholes.py` maintenant (parité call-put, cas limites comme $\text{vol} \rightarrow 0$) plutôt que d'attendre la fin du projet.

Je t'expliquerai la formule étape par étape (pourquoi $d1$, pourquoi la loi normale cumulée, pourquoi l'actualisation) et pourquoi inverser Black-Scholes n'a pas de solution fermée (d'où Newton-Raphson).

Phase 2 — Payoff et visualisation

Objectif : visualiser ce que gagne ou perd l'acheteur d'une option à maturité.

Contenu de `pricing/payoff.py` :

1. Fonction de payoff d'un call à maturité sur une gamme de prix du sous-jacent.
2. Fonction équivalente pour un put.
3. Un premier graphique `matplotlib` en local, avant toute intégration à une app.

Simple techniquement, mais tu dois pouvoir dessiner ce payoff au tableau blanc sans hésiter.

Phase 3 — Les Greeks 📌 *tu en es à la moitié*

Objectif : calculer les sensibilités du prix de l'option.

Contenu de `greeks/greeks.py` :

1. Delta, gamma, vega, theta, rho — formules analytiques dérivées de Black-Scholes.
2. Delta par différence finie (choc du spot de $+0.01$, recalcul du prix, différence divisée par le choc) — la double méthode analytique vs numérique est exactement le type de rigueur apprécié en entretien quant/trading.
3. Un test qui compare les deux méthodes.

Avant de passer à la Phase 4 : termine `tests/test_greeks.py` (comparaison analytique/numérique) — c'est le bon moment, tant que le sujet est encore frais.

Je t'expliquerai pourquoi le delta numérique doit être proche de l'analytique, et le sens de chaque Greek en langage simple, pour que tu puisses le reformuler avec tes propres mots.

Phase 4 — Monte Carlo pour l'autocall

Objectif : comprendre pourquoi un produit à barrière ne se valorise pas en formule fermée, et simuler des trajectoires.

Contenu de `monte_carlo/autocall.py` :

1. Génération de trajectoires du sous-jacent par mouvement brownien géométrique — même modèle que Black-Scholes, mais simulé pas à pas.
2. Vérification à chaque date d'observation du franchissement de la barrière de rappel anticipé.
3. Calcul du coupon si la barrière de protection tient, ou de la perte en capital sinon.
4. Moyenne actualisée sur un grand nombre de simulations (ex. 50 000) pour estimer le prix.
5. Statistiques annexes : probabilité de rappel anticipé, probabilité de perte en capital.

Point de vigilance technique : vectorise dès le départ avec `numpy` (génère toute la matrice de chocs `np.random.normal(size=(n_trajectoires, n_pas))` d'un coup, puis `cumprod/cumsum`) plutôt que des boucles `for` imbriquées — plus rapide, et une compétence `numpy` directement valorisable ("j'ai vectorisé pour éviter une boucle $O(n \times m)$ ").

On avance lentement : d'abord 100 trajectoires pour vérifier visuellement, puis on monte en puissance. Je t'expliquerai pourquoi le nombre de simulations influence la précision, et pourquoi ce produit est dit "path-dependent".

Test immédiat : `tests/test_autocall.py` — vérifier que le prix converge quand le nombre de simulations augmente.

Pourquoi l'autocall en particulier : c'est un produit signature des desks structurés français (SG, BNP Paribas, Natixis en sont des acteurs historiques) — un bon choix pour ta cible, à mentionner explicitement dans le README.

Phase 5 — Scénarios de marché et stress test

Objectif : mesurer l'impact d'un choc de marché — compétence directement utile à un desk.

Contenu de `risk/scenarios.py` :

1. Fonction qui reprix une option ou un autocall après un choc du spot (ex. $\pm 10\%$).
2. Fonction qui reprix après un choc de volatilité (ex. +5 points de vol).
3. Un tableau récapitulatif : prix avant, prix après, variation en euros et en %.

Phase 6 — Hedge & P&L Explain

Objectif : la fonctionnalité la plus différenciante du projet.

Contenu de `risk/pnl_explain.py` :

1. P&L "réel" : reprix complet avant/après un mouvement de marché simulé.

2. P&L "estimé" via les Greeks (formule de Taylor : $\Delta \cdot \Delta S + \frac{1}{2} \cdot \Gamma \cdot \Delta S^2 + \Theta \cdot \Delta t + \text{Vega} \cdot \Delta \sigma + \rho \cdot \Delta r$).
3. Comparaison des deux, avec un texte explicatif selon l'ampleur de l'écart (petit mouvement → écart faible ; gros mouvement → écart plus grand à cause de la convexité).

C'est la section à défendre en profondeur : elle prouve que tu comprends la différence entre repricing exact et approximation locale — un trader ou un risk manager la questionnera en priorité, et c'est littéralement ce que font les desks au quotidien pour réconcilier P&L réalisé et P&L attendu.

Phase 7 — Consolidation des tests

Changement par rapport à la v1 : la majorité des tests sont déjà écrits au fil des phases précédentes. Cette phase sert à compléter, pas à démarrer :

1. Vérifier la couverture des cas limites (vol=0, maturité très courte, spot = strike exactement).
 2. Ajouter le test de parité call-put s'il manque encore.
 3. Vérifier que tout tourne avec `pytest` en une seule commande, proprement.
-

Phase 8 — Interface Streamlit + déploiement

Objectif : rendre le tout utilisable visuellement.

Contenu de `app.py` :

1. Barre latérale pour les paramètres (spot, strike, maturité, volatilité, taux, type de produit).
2. Onglet "Pricing & Greeks" (Black-Scholes + volatilité implicite).
3. Onglet "Autocall Monte Carlo".
4. Onglet "Scénarios de marché".
5. Onglet "Hedge & P&L Explain".

Ajout par rapport à la v1 : déploie l'app sur Streamlit Community Cloud (gratuit, ~10 min une fois le repo prêt). Un lien cliquable vers une app qui tourne a beaucoup plus d'impact sur un CV/LinkedIn qu'un lien GitHub que le recruteur doit cloner lui-même — en pratique, presque personne ne le fera.

Phase 9 — Documentation et présentation

Objectif : donner au projet une vraie crédibilité de portfolio.

1. `README.md` — **structure importante** : en haut, une accroche en 1-2 phrases, une capture d'écran ou un GIF de l'app, et le lien vers l'app déployée. En dessous seulement : objectif détaillé, formules, hypothèses, limites du modèle (vol constante, autocall simplifié, pas de coûts de transaction).
 2. Une section "Ce que j'ai appris", rédigée avec tes propres mots.
 3. Une courte vidéo ou GIF de démo.
-

Phase 10 — Préparation aux questions d'entretien

On listera ensemble les questions probables : pourquoi Black-Scholes, pourquoi Monte Carlo pour l'autocall, ce que signifie chaque Greek, pourquoi le P&L estimé diffère du P&L réel.

Ajout par rapport à la v1 : prépare aussi la question inverse, souvent posée par un interviewer senior pour tester ta maturité plus que tes connaissances — *"Qu'est-ce que ton modèle ne fait pas qu'un vrai pricer de desk ferait ?"* (vol non constante en pratique/smile, calibration sur prix cotés, Greeks recalculés en temps réel, funding/XVA, barrière observée en continu vs discrète). Savoir citer ces limites sans les avoir codées est souvent plus impressionnant que le code lui-même.

Bonus optionnel — Confrontation aux données de marché réelles

Si le temps le permet une fois la Phase 6 terminée : avec `yfinance` (gratuit), récupère le spot d'une action ou d'un indice réel, compare ton prix Black-Scholes théorique à un prix d'option réellement coté. Transforme un projet "académique" en projet confronté au marché réel, et illustre concrètement les limites du modèle (écart = smile de vol, dividendes mal estimés...).

Checklist finale

- ☐ Phase 0 — repo + venv + premier commit minimal
- ☐ Phase 1 — Black-Scholes + volatilité implicite + test
- ☐ Phase 2 — Payoff + visualisation
- ☐ Phase 3 — Greeks (analytique + numérique) + test (*en cours*)
- ☐ Phase 4 — Monte Carlo autocall (vectorisé) + test
- ☐ Phase 5 — Scénarios de marché
- ☐ Phase 6 — Hedge & P&L Explain
- ☐ Phase 7 — Consolidation des tests
- ☐ Phase 8 — App Streamlit + déploiement public
- ☐ Phase 9 — README (accroche + screenshot en haut) + démo
- ☐ Phase 10 — Prep entretien (y compris limites du modèle)
- ☐ Bonus — Comparaison à un prix de marché réel (optionnel)

